



KANTON SARAJEVO

MINISTARSTVO ZA ODGOJ I OBRAZOVANJE
KANTONA SARAJEVO

Kantonalno takmičenje iz informatike za srednje škole

Bilten pripremili:

Hadžić Hadžem

Zavlan Amer

Džaka Tarik

Parić Muamer

12. Mart 2026. GODINE

Rezultati	3
Poredak takmičara	3
Poredak škola	5
Postavke zadataka	6
Zadatak 1 - NZD	6
Zadatak 2 - Jačina šifre	9
Zadatak 3 - Biblioteka	15
Zadatak 4 - Skakutanje	19
Zadatak 5 - Penjanje	22
Rješenja zadataka	27
Zadatak 1 - Portal	27
Zadatak 2 - Jačina šifre	28
Zadatak 3 - Biblioteka	31
Zadatak 4 - Skakutanje	32
Zadatak 5 - Penjanje	34



Rezultati

Poredak takmičara

Ime i prezime	Škola	Z1	Z2	Z3	Z4	Z4	Ukupno
Mirza Bušatlić	Druga gimnazija Sarajevo	100	100	100	100	61	461
Adnan Osmić	Druga gimnazija Sarajevo	100	100	100	50	61	411
Faruk Demirović	JU Peta gimnazija	100	100	100	50	32	382
Amir Karamović	Druga gimnazija Sarajevo	100	100	100	50	0	350
Farah Demirović	JU Peta gimnazija	100	100	100	50	0	350
Omer Drkić	JU Gazi Husrev-begova medresa u Sarajevu	100	100	100	0	0	300
Tarik Dacić	Druga gimnazija Sarajevo	100	100	100	0	0	300
Mak Nurboja	Druga gimnazija Sarajevo	100	67.5	46	50	0	263.5
Tarik Odžak	Treća gimnazija	100	41.5	19	40	0	200.5
Emir Salčinović	Treća gimnazija	80	50	57	0	0	187
Faris Šabeta	Druga gimnazija Sarajevo	100	30.5	0	0	0	130.5
Ismail Bektešević	Srednjoškolski centar "Nedžad Ibrišimović" Ilijaš	80	50	0	0	0	130
Emrah Isović	JU Srednja mašinska tehnička škola	100	9.5	19	0	0	128.5
Harun Hubijar	JU Četvrta gimnazija Ilidža	80	42	0	0	0	122
Halid Džunuzović	Srednja elektrotehnička škola Sarajevo	80	32	8	0	0	120
Tarik Tatarević	Srednjoškolski centar "Nedžad Ibrišimović" Ilijaš	50	69.5	0	0	0	119.5
Marko Dragić	Katolički školski centar - Opća-realna gimnazija	100	0	8	0	0	108
Dominik Dzolan	Katolički školski centar - Opća-realna	100	0	8	0	0	108



	gimnazija						
Borna Mraović	JU GIMNAZIJA DOBRINJA - SARAJEVO	80	26	0	0	0	106
Fatih Omerović	Treća gimnazija	50	45.5	8	0	0	103.5
Andrej Đaković	Katolički školski centar - Opća-realna gimnazija	100	0	0	0	0	100
Ahmed Čengić	Srednjoškolski centar "Nedžad Ibrišimović" Ilijaš	80	20	0	0	0	100
Filip Tomić	Javna ustanova GIMNAZIJA OBALA Sarajevo	80	16.5	0	0	0	96.5
Benjamin Hrvo	Srednja elektrotehnička škola Sarajevo	80	3.5	0	0	0	83.5
Ilhan Suljević	Prva bošnjačka gimnazija	80	0	0	0	0	80
Mustafa Alijagić	Perzijsko-bosanski koledž sa internatom	80	0	0	0	0	80
Ajnur Šahić	Srednjoškolski centar Vogošća	80	0	0	0	0	80
Emil Macić	JU Četvrta gimnazija Ilidža	80	0	0	0	0	80
Ahmed Kurtović	JU Četvrta gimnazija Ilidža	80	0	0	0	0	80
Hamza Hajlovac	Perzijsko-bosanski koledž sa internatom	80	0	0	0	0	80
Imad Džigal	Srednjoškolski centar Vogošća	50	18	0	0	0	68
Zlatan Hadžić	JU Srednja ekonomska škola, Sarajevo	50	0	0	0	0	50
Tarik Terzić	JU Prva gimnazija Sarajevo	10	0	0	0	0	10
Amsal Kadrić	JU Prva gimnazija Sarajevo	10	0	0	0	0	10
Din Taletović	JU GIMNAZIJA DOBRINJA - SARAJEVO	0	0	0	0	0	0
Mak Žiga	Javna ustanova GIMNAZIJA OBALA Sarajevo	0	0	0	0	0	0
Emir Mulahusić	Gymnasium Sarajevo School of Science and Technology	0	0	0	0	0	0
Nikola Kujundžija	Javna ustanova GIMNAZIJA OBALA Sarajevo	0	0	0	0	0	0
Enver Operta	Srednja elektrotehnička škola Sarajevo	0	0	0	0	0	0
Imran Džindo	JU Gazi Husrev-begova medresa u Sarajevu	0	0	0	0	0	0



Mak Mehmedić	JU Srednja mašinska tehnička škola	0	0	0	0	0	0
Amna Pačo	Srednjoškolski centar Vogošća	0	0	0	0	0	0
Tarik Hromo	JU Prva gimnazija Sarajevo	0	0	0	0	0	0
Harun Seljubac	Prva bošnjačka gimnazija	0	0	0	0	0	0
Eldin Zumberi	JU Srednja mašinska tehnička škola	0	0	0	0	0	0

Poredak škola

Redno mjesto	Škola	Ukupni zbir poena
1	Druga gimnazija Sarajevo	1222
2	JU Peta gimnazija	732
3	JU Gazi Husrev-begova medresa u Sarajevu	300
4	Treća gimnazija	491
5	Srednjoškolski centar "Nedžad Ibrišimović" Ilijaš	349.5
6	JU Srednja mašinska tehnička škola	128.5
7	JU Četvrta gimnazija Ilidža	282
8	Srednja elektrotehnička škola Sarajevo	203.5
9	Katolički školski centar - Opća-realna gimnazija	316
10	JU GIMNAZIJA DOBRINJA - SARAJEVO	106
11	Javna ustanova GIMNAZIJA OBALA Sarajevo	96.5
12	Prva bošnjačka gimnazija	80
13	Perzijsko-bosanski koledž sa internatom	160
14	Srednjoškolski centar Vogošća	148
15	JU Srednja ekonomska škola, Sarajevo	50
16	JU Prva gimnazija Sarajevo	20
17	Gymnasium Sarajevo School of Science and Technology	0



Postavke zadatka

Zadatak 1 - NZD

Krenimo sa jednostavnim zadatkom. Za unesene prirodne brojeve A i B potrebno je odrediti najveći broj kojim su djeljivi A i B . Drugim riječima, potrebno je odrediti njihov najveći zajednički djelilac (NZD). Obratite pažnju na to da brojevi mogu biti vrlo veliki.

Sretno takmičenje!

Ulazni podaci

U jednom redu ulaza se nalaze dva prirodna broja A i B , odvojena razmakom.

Ograničenja

$$1 \leq A, B \leq 10^{18}$$

Podzadaci

Podzadatak 1 (10 bodova)

$$1 \leq A, B \leq 100$$

A i B su međusobno prosti.

Podzadatak 2 (40 bodova)

$$1 \leq A, B \leq 100$$



Podzadatak 3 (30 bodova)

$$1 \leq A, B \leq 10^9$$

Podzadatak 4 (20 bodova)

Bez dodatnih ograničenja.

Izlazni podaci

U jedini red izlaza ispišite jedan prirodan broj: najveći zajednički djelilac brojeva A i B .

Primjeri

Ulaz 1

12 8

Izlaz 1

4

Objašnjenje 1

Djelioci broja 12 su: 1, 2, 3, 4, 6, 12. Djelioci broja 8 su: 1, 2, 4, 8. Zajednički djelioci su 1, 2, 4, a najveći među njima je 4.

Ulaz 2

7 13

Izlaz 2

1



Objašnjenje 2

Brojevi 7 i 13 su prosti brojevi, pa im je jedini zajednički djelilac 1. Dakle, $\text{NZD}(7, 13) = 1$.

Ulaz 3

100 25

Izlaz 3

25

Objašnjenje 3

Broj 25 dijeli broj 100 ($100 = 4 \cdot 25$), pa je najveći zajednički djelilac jednak 25.



Zadatak 2 - Jačina šifre

Data vam je lista šifri. Za svaku šifru odredite njenu **jačinu**.

Rezultat jačine je cijeli broj od **0 do 7**, izračunat pomoću **7 nezavisnih pravila**. Svako zadovoljeno pravilo donosi **1 bod**, a zbir svih bodova nam daje **jačinu šifre**.

Šifra je niz znakova dužine **od 1 do 20** znakova (bez razmaka).

Važna napomena o parcijalnim rješenjima

Nije obavezno implementirati sva pravila da biste dobili bodove.

- Možete implementirati **bilo koji podskup pravila**.
- Pravila su **nezavisna** - na primjer, možete implementirati **samo Pravilo 2 i Pravilo 4**, bez ostalih.
- Vaše rješenje će biti **parcijalno bodovano**, proporcionalno broju pravila koja ste ispravno implementirali.
- Općenito, **što više pravila ispravno implementirate, veći će biti vaš rezultat**.

Pogledajte odjeljak **Bodovanje** za više informacija.

Pravila

Neka je s šifra, a $n = \text{duzina}(s)$.



Pravilo 1 - Minimalna dužina

Šifra ima **najmanje 11 znakova**.

Primjeri

- Zadovoljava: abcdefghijk
 - Ne zadovoljava: abc123!
-

Pravilo 2 - Malo slovo na početku

Prvi znak je malo slovo engleske abecede ($a - z$).

Primjeri

- Zadovoljava: aBCdef12
 - Ne zadovoljava: Abcdef12
-

Pravilo 3 - Cifra na kraju

Posljednji znak je cifra (0-9).

Primjeri

- Zadovoljava: Abcdef!5
 - Ne zadovoljava: Abcdef!x
-

Pravilo 4 - Specijalni znak na drugom ili predzadnjem mjestu

Drugi ili pretposljednji znak je jedan od znakova iz skupa specijalnih karaktera: !@#\$%&*?_

Primjeri



- Zadovoljava (drugi znak je specijalan): a!bcdef9
 - Zadovoljava (pretposljednji znak je specijalan): abCdef!9
 - Ne zadovoljava: abcdefg9
-

Pravilo 5 - Raznovrsnost znakova

Šifra sadrži znakove iz **najmanje 3 od sljedeće 4 kategorije**:

- neko od navedenih malih slova engleske abecede (abcde)
- neko od navedenih velikih slova engleske abecede (ABCDE)
- neku od navedenih cifara (01234)
- neki od navedenih specijalnih znakova: !@#

Primjeri

- Zadovoljava: aB3xxxx (mala slova, velika slova, cifre)
 - Ne zadovoljava: abcdefgh (samo mala slova)
-

Pravilo 6 - Mnogo ponavljanja

Postoji **barem jedan znak** koji se pojavljuje **4 ili više puta** u šifri.

Primjeri

- Zadovoljava: aaaabC1! (*a* se ponavlja 4 puta)
 - Zadovoljava: aaCaba1! (*a* se ponavlja 4 puta)
 - Ne zadovoljava: aaabC1!
-

Pravilo 7 - Ponovljeni bigram (bez preklapanja)

Lozinka sadrži **ponovljeni podniz dužine 2** (tzv. *bigram*) tako da se ta dva pojavljivanja ne preklapaju.



Primjeri

- Zadovoljava: abXYabZ9 (bigram ab se pojavljuje dva puta)
 - Ne zadovoljava: abcdefg9
 - Ne zadovoljava: aaadefg9 (iako imamo bigrame aa koji se nalaze na poziciji **0** i **1**, oni se preklapaju)
-

Ulaz

Prvi red sadrži cijeli broj T - broj šifri. Svaki od narednih T redova sadrži jednu šifru.

Izlaz

Za svaku šifru ispišite jedan cijeli broj - njen rezultat jačine.

Ograničenja

$$1 \leq T \leq 200$$

$$1 \leq \text{duzina(lozinke)} \leq 20$$

Primjer

Ulaz

5

abc123!

a!B3xxxx9ZZ

Abcdef!9



aBCdef12
a!a!B1B2XX

Izlaz

2
6
3
3
4

Objašnjenje

Lozinka	Pr. 1	Pr. 2	Pr. 3	Pr. 4	Pr. 5	Pr. 6	Pr. 7	Rez.
abc123!		✓			✓			2
a!B3xxxx9ZZ	✓	✓		✓	✓	✓	✓	6
Abcdef!9			✓	✓	✓			3
aBCdef12		✓	✓		✓			3
a!a!B1B2XX		✓		✓	✓		✓	4

Detalji:

- abc123!: P2 — počinje sa **a**; P5 — mala slova + cifre + specijalni
- a!B3xxxx9ZZ: P1 — dužina ≥ 11 ; P2 — počinje sa **a**; P4 — drugi znak **!**; P5 — sve kategorije; P6 — **x** $\times 4$; P7 — bigram **xx**
- Abcdef!9: P3 — završava sa **9**; P4 — pretposljednji **!**; P5 — mala + velika + specijalni

Kantonalno takmičenje iz informatike

Srednje škole

Kanton Sarajevo, 2026



- aBCdef12: P2 — počinje sa **a**; P3 — završava sa **2**; P5 — mala + velika + cifre
- a!a!B1B2XX: P2 — počinje sa **a**; P4 — drugi znak **!**; P5 — sve kategorije; P7 — bigram **a!**



Zadatak 3 - Biblioteka

Indira je strastvena ljubiteljica knjiga i u svom domu ima impresivnu kućnu biblioteku u kojoj se nalazi N knjiga. Sve knjige na policama pažljivo su poredane u neopadajućem redoslijedu prema njihovim visinama, mjeranim u milimetrima. Ovaj sistem organizacije omogućava joj da lako pronađe bilo koju knjigu i čini njenu biblioteku estetski privlačnom.

Nedavno je Indira poručila novu kolekciju od M knjiga iz antikvarijata. Kada su knjige stigle, shvatila je da mora organizovati prostor na policama kako bi dodala sve nove knjige, pritom održavajući neopadajući redoslijed visina.

Dodavanje novih knjiga zahtijeva fizički napor, pošto mora razmicati postojeće knjige kako bi napravila prostor za nove. Indira želi minimizirati broj **mjesta** na kojima će morati razmicati knjige. **Mjesto** predstavlja poziciju između dvije postojeće knjige (ili na početku ili kraju police) gdje će se dodati jedna ili više novih knjiga. Na primjer, ako Indira doda 3 nove knjige između iste dvije postojeće knjige, to se računa kao samo jedno mjesto, jer je samo jednom morala razmicati knjige da napravi prostor.

Pomozite Indiri da odredi minimalni broj mjesta na kojima će morati razmicati knjige kako bi dodala sve nove knjige i održala neopadajući redoslijed.

Ulazni podaci

U prvom redu ulaza se nalazi prirodan broj N , broj knjiga koje Indira trenutno ima u biblioteci.

U drugom redu ulaza se nalazi niz A dužine N , koji predstavlja visine postojećih knjiga u milimetrima, poredane u neopadajućem redoslijedu. Elementi niza su odvojeni razmacima. Konkretno, $A[i]$ predstavlja visinu i -te knjige.

U trećem redu ulaza se nalazi prirodan broj M , broj novih knjiga koje Indira treba dodati.



U četvrtom redu ulaza se nalazi niz B dužine M , koji predstavlja visine novih knjiga u milimetrima, poredane u neopadajućem redoslijedu. Elementi niza su odvojeni razmacima. Konkretno, $B[j]$ predstavlja visinu j -te nove knjige.

Ograničenja

$$1 \leq N, M \leq 10^5$$

$$1 \leq A[i], B[j] \leq 10^9$$

Podzadaci

Podzadatak 1 (8 bodova)

$$1 \leq N, M, A[i], B[j] \leq 10$$

Visine svih novih knjiga su iste, odnosno $B[0] = B[1] = \dots = B[M - 1]$.

Podzadatak 2 (11 bodova)

$$1 \leq M, A[i], B[j] \leq 10$$

$$N = 1$$

Podzadatak 3 (38 bodova)

$$1 \leq N, M, A[i], B[j] \leq 10$$

Podzadatak 4 (35 bodova)

$$1 \leq N, M, A[i], B[j] \leq 1000$$

Podzadatak 5 (8 bodova)

Bez dodatnih ograničenja.



Izlazni podaci

U jedini red izlaza ispišite jedan prirodan broj: minimalni broj mjesta na kojima Indira mora razmicati knjige kako bi dodala sve nove knjige i održala neopadajući redoslijed visina.

Primjer

Ulaz 1

4

100 150 200 250

3

120 180 190

Izlaz 1

2

Objašnjenje 1

Indira trenutno ima 4 knjige sa visinama 100, 150, 200 i 250 milimetara. Potrebno je dodati 3 nove knjige sa visinama 120, 180 i 190 milimetara. Optimalno rješenje je sljedeće:

- Knjiga visine 120 mm se dodaje između knjiga visine 100 i 150.
- Knjige visine 180 i 190 mm se obje dodaju između knjiga visine 150 i 200.

Konačni poredak knjiga po visinama bit će: 100, 120, 150, 180, 190, 200, 250.

Indira je morala razmicati knjige na samo 2 različita mjesta, što je minimalan broj potrebnih mjesta.

Ulaz 2

3

100 500 1000

4

50 400 500 1500

Izlaz 2

3



Objašnjenje 2

Indira trenutno ima 3 knjige sa visinama 100, 500 i 1000 milimetara. Potrebno je dodati 4 nove knjige sa visinama 50, 400, 500 i 1500 milimetara.

Optimalno rješenje je sljedeće:

- Knjiga visine 50 mm se dodaje na početak police, prije knjige visine 100.
- Knjige visine 400 i 500 mm se obje dodaju između knjiga visine 100 i 500.
- Knjiga visine 1500 mm se dodaje na kraj police, poslije knjige visine 1000.

Konačni poredak knjiga po visinama bit će: 50, 100, 400, 500, 500, 1000, 1500.

Indira je morala razmicati knjige na 3 različita mjesta, što je minimalan broj potrebnih mjesta.

Ulaz 3

5

10 10 20 50 100

2

20 50

Izlaz 3

1

Objašnjenje 3

Indira trenutno ima 5 knjiga sa visinama 10, 10, 20, 50 i 100 milimetara. Potrebno je dodati 2 nove knjige sa visinama 20 i 50 milimetara.

Optimalno rješenje je da obje nove knjige dodamo između knjiga visine 20 i 50. Na ovaj način razmičemo knjige samo na jednom mjestu.

Konačni poredak knjiga po visinama bit će: 10, 10, 20, 20, 50, 50, 100.

Indira je morala razmicati knjige na samo 1 mjestu, što je minimalan broj potrebnih mjesta.



Zadatak 4 - Skakutanje

Muki je razvio novu vrstu igre školice. U ovoj igri, red kamenja, prigodno označenih brojevima od 1 do N (uključivo), postavljen je u bazen lave. Kao što se može očekivati, kamenje je veoma toplo. Zapravo, može biti toplije i od površine Sunca!

Muki mora preskočiti sa početka (pozicija 0) na drugu stranu (pozicija $N + 1$), ali u jednom skoku može preskočiti najviše K kamenova unaprijed. Srećom, Muki ima par rashladnih čizama koje mogu ohladiti bilo koji kamen na 0 stepeni. Međutim, čizme zahtijevaju jednu jedinicu energije za svaki stepen hlađenja, a energija je veoma skupa!

Možete li pomoći Mukiju da pronađe minimalnu količinu energije potrebnu za prelazak na drugu stranu bazena lave?

- Muki može stati samo na kamenje koje je ohlađeno na 0 stepeni.
- Muki može skočiti unaprijed samo na kamen j ($i < j \leq i + K$) sa kamena i .
- Budući da Muki kreće sa pozicije 0, u svom prvom skoku može skočiti na kamen j ($0 < j \leq K$).

Sretno takmičenje!

Ulazni podaci

U prvom redu ulaza se nalaze dva prirodna broja N i K , odvojena razmakom.

U drugom redu ulaza nalazi se N nenegativnih cijelih brojeva n_i ($1 \leq i \leq N$) odvojenih razmacima, koji predstavljaju temperature i -tog kamena.

Ograničenja

$$1 \leq N \leq 2^{23}$$

$$1 \leq K \leq N$$



$$0 \leq n_i < 2^{63}$$

$$0 \leq \sum_{i=1}^N n_i < 2^{63}$$

Podzadaci

Podzadatak 1 (10 bod)

$$N \leq 1000$$

Podzadatak 2 (10 bod)

$$K = N$$

Podzadatak 3 (30 boda)

$$K = 1$$

Podzadatak 4 (50 bodova)

Bez dodatnih ograničenja.

Izlazni podaci

U prvi red izlaza ispišite jedan broj: minimalan utrošak energije za skok sa pozicije 0 do $N + 1$.

U drugi red izlaza ispišite indekse kamenova na koje Muki mora skočiti kako bi iskoristio minimalnu količinu energije, odvojene razmacima i u rastućem redoslijedu. Ako postoji više načina da postigne minimalnu energiju, ispišite **leksikografski najveći** niz indeksa.

Primjeri

Ulaz 1

5 3

3 2 5 2 4



Izlaz 1

4
2 4

Objašnjenje 1

Muki počinje na poziciji 0. Skače na kamen 2, hladi ga za 2 stepena na 0 (utrošena energija: 2). Sa kamena 2 skače na kamen 4, hladi ga za 2 stepena na 0 (utrošena energija: 2). Sa kamena 4 skače na drugu stranu bazena lave (pozicija 6). Ukupna utrošena energija je $2 + 2 = 4$, a indeksi kamenova su 2 i 4. Ostali putevi rezultiraju jednakom ili većom potrošnjom energije.

Ulaz 2

4 2
1 2 1 1

Izlaz 2

2
1 3

Objašnjenje 2

Najjeftiniji put za Mukija je da sa pozicije 0 skoči na kamen 1 (energija: 1), a zatim sa kamena 1 na kamen 3 (energija: 1). Sa kamena 3 može direktno skočiti na poziciju 5. Ukupna energija je 2, a odabrani indeksi su 1 i 3.

Ulaz 3

4 2
2 2 2 2

Izlaz 3

4
2 4

Objašnjenje 3

Svi kamenovi imaju istu temperaturu. Moguće je preći bazen s potrošnjom od 4 jedinice energije na više načina (npr. posjećivanjem kamenova 1 3, 2 3 ili 2 4). Od svih nabrojanih rješenja, niz 2 4 je leksikografski najveći, pa stoga ispisujemo njega.



Zadatak 5 - Penjanje

Jednog lijepog dana, Una i Jelena odlučuju da odu na penjanje po stijeni. Stijena za penjanje ima N hvatova (izbočina za hvatanje). Svaki hvat i se nalazi na visini Y_i od dna zida i X_i udaljena od centra zida u horizontalnom smjeru. Ako je X_i negativan, hvat se nalazi lijevo od centra, a ako je pozitivan, hvat se nalazi desno od centra. Svi hvatovi imaju različite pozicije.

Da bi testirala Jelenine vještine penjanja, Una je odlučila postaviti joj izazov.

Opis izazova

Izazov funkcionira na sljedeći način:

1. Od N hvatova, Una će izabrati skup od K hvatova. Ovaj skup ćemo označiti sa R .
2. Da bi pobijedila u izazovu, Jelena prvo mora izabrati jedan par hvatova (A, B) iz skupa R . Jelena može izabrati bilo koji par hvatova sve dok je $A \neq B$ i oba hvata su u skupu R .
3. Jelena će početi od prvog hvata (A) i pokušati da stigne do drugog hvata (B) . Ona može prolaziti kroz druge hvatove na putu od A do B , bez obzira da li je taj hvat u skupu R ili ne.
4. Međutim, svaki hvat ima nivo trenja S_i . Kada hvat ima nizak nivo trenja, teže je doseći dalek hvat bez padanja. Dodatno, Una dozvoljava Jeleni da se penje samo naviše. Preciznije, da bi se pomaknula sa hvata i na hvat j , mora biti ispunjeno: $\max(|X_i - X_j|, |Y_i - Y_j|) \leq \max(S_i, S_j)$ i $Y_i < Y_j$.
5. Jelena će pobijediti u izazovu ako uspije da pronađe bar jedan par hvatova (A, B) takav da može stići od hvata A do hvata B . Ako ne uspije u tome, Jelena gubi izazov.

Naravno, Jelena zna da postoji mnogo parova hvatova za koje se izazov ne može završiti. Ona želi da pronađe minimalno K takvo da bez obzira koji skup hvatova Una izabere, ona uvijek može završiti izazov. Potrebna joj je vaša pomoć da pronađe tu vrijednost.



Ulazni podaci

U prvom redu ulaza se nalazi prirodan broj N , broj hvatova na zidu.

U sljedećih N redova se nalaze po tri cijela broja. Red $(i + 1)$ sadrži brojeve X_i , Y_i i S_i koji predstavljaju horizontalnu poziciju, visinu i nivo trenja i -tog hvata.

Ograničenja

$$1 \leq N \leq 500$$

$$-10^6 \leq X_i \leq 10^6$$

$$1 \leq Y_i \leq 10^6$$

$$1 \leq S_i \leq 2 \times 10^6$$

Podzadaci

Podzadatak 1 (15 bodova)

$$1 \leq N \leq 20$$

$$S_i = 2 \times 10^6 \text{ za sve } i$$

Podzadatak 2 (29 bodova)

$$1 \leq N \leq 20$$

Bez dodatnih ograničenja

Podzadatak 3 (17 bodova)

$$1 \leq N \leq 500$$

$$X_i = 0 \text{ za sve } i$$



$$S_i = S_j \text{ za sve } i, j$$

Podzadatak 4 (19 bodova)

$$1 \leq N \leq 500$$

$$S_i = 1 \text{ za sve } i$$

Y_i nije djeljivo sa 3 za sve i

Podzadatak 5 (20 bodova)

$$1 \leq N \leq 500$$

Bez dodatnih ograničenja

Izlazni podaci

U jedini red izlaza ispišite jedan cijeli broj: minimalni broj hvatova koje Una mora izabrati kako bi Jelena uvijek mogla završiti izazov. Ako Jelena nikada ne može završiti izazov, ispišite -1 .

Primjeri

Ulaz 1

5
0 3 2
-1 5 1
4 4 3
-1 1 2
2 2 1

Izlaz 1

3



Objašnjenje 1

Kada je $K = 2$, postoji neki podskup hvatova tako da Jelena ne može završiti izazov. Na primjer, ako Una izabere hvatove na pozicijama $(-1, 1)$ i $(2, 2)$ da formiraju skup R , Jelena ne može završiti izazov.

Jelena se ne može pomaknuti sa hvata na $(2, 2)$ do hvata na $(-1, 1)$ jer se može penjati samo naviše. Jelena se ne može direktno pomaknuti sa hvata na $(-1, 1)$ do hvata na $(2, 2)$ jer je $\max(|-1 - 2|, |1 - 2|) = 3 > \max(2, 1) = 2$.

Druga opcija je da se pomakne indirektno sa $(-1, 1)$ na $(2, 2)$. Jelena može da se popne sa hvata na $(-1, 1)$ na hvat na $(0, 3)$ jer je $\max(|-1 - 0|, |1 - 3|) = 2 \leq \max(2, 1) = 2$. Međutim, ona se ne može pomaknuti sa hvata na $(0, 3)$ na hvat na $(2, 2)$ jer mora da se penje naviše.

Može se provjeriti da za svaki skup od 3 hvata, uvijek postoji način da se izabere par hvatova tako da Jelena može završiti izazov.

Na primjer, ako Una izabere hvatove na pozicijama $(-1, 5)$, $(4, 4)$, $(-1, 1)$ da formiraju skup R , Jelena može završiti izazov tako što će izabrati hvatove na $(-1, 5)$ i $(-1, 1)$.

Da bi završila izazov, ona se penje sa hvata $(-1, 1)$ na hvat $(0, 3)$, a zatim na hvat $(-1, 5)$. Posredni hvatovi ne moraju biti iz skupa R .

Ulaz 2

3

0 1 2000000

-1 2 2000000

1 2 2000000

Izlaz 2

3



Objašnjenje 2

Ako Una izabere 2 hvata na visini 2 (tj. drugi i treći хват), Jelena ne može završiti izazov jer se može penjati samo naviše, a oba hvata su na istoj visini. Međutim, ako Una izabere sva 3 hvata, Jelena uvijek može izabrati par koji uključuje хват na visini 1, od kojeg može stići do bilo kojeg hvata na visini 2. Zato je minimalno $K = 3$.

Ulaz 3

2

0 1 1

0 5 1

Izlaz 3

-1

Objašnjenje 3

Čak i ako Una izabere sve hvatove, ne postoji način da Jelena stigne sa jednog hvata do drugog. Zato Jelena nikada ne može završiti izazov.



Rješenja zadataka

Sva rješenja, tekstovi zadataka kao i testni slučajevi su (ili će biti) dostupni na linku: https://github.com/hhadzem/ks_inf_takm.

Zadatak 1 - Portal

```
#include <iostream>

using namespace std;

int main()
{
    long long A, B;
    cin >> A >> B;

    long long temp;

    while (B != 0)
    {
        temp = B;
        B = A % B;
        A = temp;
    }

    cout << A << endl;

    return 0;
}
```



Zadatak 2 - Jačina šifre

```
#include <iostream>
#include <string>
#include <vector>
#include <cctype>

#define NUMBER_OF_RULES 7
#define RULE_1_LENGTH 11

bool rules_activated[NUMBER_OF_RULES];

void reset_rules_array() {
    for (int i = 0; i < NUMBER_OF_RULES; i++) {
        rules_activated[i] = false;
    }
}

bool rule1(const std::string& s) {
    return s.length() >= RULE_1_LENGTH;
}

bool rule2(const std::string& s) {
    if (s.empty()) return false;
    return std::islower(s[0]);
}

bool rule3(const std::string& s) {
    if (s.empty()) return false;
    return std::isdigit(s[s.length() - 1]);
}

bool rule4(const std::string& s) {
    if (s.length() < 2) return false;

    char second = s[1];
    if (second == '!' || second == '@' || second == '#' || second == '$' ||
        second == '%' || second == '&' || second == '*' || second == '?' ||
        second == '_') {
        return true;
    }
}

char second_last = s[s.length() - 2];
if (second_last == '!' || second_last == '@' || second_last == '#' || second_last == '$'
```



```

||
    second_last == '%' || second_last == '&' || second_last == '*' || second_last == '?'
||
    second_last == '_' ) {
        return true;
    }

    return false;
}

bool rule5(const std::string& s) {
    bool has_lower = false;
    bool has_upper = false;
    bool has_digit = false;
    bool has_special = false;

    for (char c : s) {
        if (c >= 'a' && c <= 'e') {
            has_lower = true;
        } else if (c >= 'A' && c <= 'E') {
            has_upper = true;
        } else if (c >= '0' && c <= '4') {
            has_digit = true;
        } else if (c == '!' || c == '@' || c == '#') {
            has_special = true;
        }
    }

    int count = (has_lower ? 1 : 0) + (has_upper ? 1 : 0) + (has_digit ? 1 : 0) +
(has_special ? 1 : 0);
    return count >= 3;
}

bool rule6(const std::string& s) {
    std::vector<int> count(256, 0);

    for (char c : s) {
        count[c]++;
        if (count[c] >= 4) {
            return true;
        }
    }
    return false;
}

bool rule7(const std::string& s) {
    if (s.length() < 4) return false;
}

```



```

for (size_t i = 0; i < s.length() - 3; i++) {
    char first_char = s[i];
    char second_char = s[i + 1];

    for (size_t j = i + 2; j < s.length() - 1; j++) {
        if (s[j] == first_char && s[j + 1] == second_char) {
            return true;
        }
    }
}
return false;
}

int calculate_password_strength(const std::string& password) {
    reset_rules_array();

    rules_activated[0] = rule1(password);
    rules_activated[1] = rule2(password);
    rules_activated[2] = rule3(password);
    rules_activated[3] = rule4(password);
    rules_activated[4] = rule5(password);
    rules_activated[5] = rule6(password);
    rules_activated[6] = rule7(password);

    int score = 0;
    for (int i = 0; i < NUMBER_OF_RULES; i++) {
        if (rules_activated[i]) {
            score++;
        }
    }
    return score;
}

int main() {
    int T;
    std::cin >> T;
    std::cin.ignore();

    for (int i = 0; i < T; i++) {
        std::string password;
        std::getline(std::cin, password);
        std::cout << calculate_password_strength(password) << std::endl;
    }

    return 0;
}

```



Zadatak 3 - Biblioteka

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int N, M;

    cin >> N;
    vector<int> A(N);
    for (int i = 0; i < N; i++) cin >> A[i];

    cin >> M;
    vector<int> B(M);
    for (int i = 0; i < M; i++) cin >> B[i];

    int i = 0; // pointer za postojeće knjige A
    int j = 0; // pointer za nove knjige B
    int br = 0; // brojač mjesta

    while (i < N && j < M) {
        if (A[i] > B[j]) {
            br++;
            while (j < M && A[i] >= B[j]) {
                j++;
            }
        } else {
            i++;
        }
    }

    if (j < M) {
        br++;
    }
    cout << br << endl;
    return 0;
}
```



Zadatak 4 - Skakutanje

```
#include <iostream>
#include <vector>
#include <deque>
using namespace std;
typedef unsigned long long ull;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, K;
    cin >> N >> K;

    vector<ull> cost(N + 2, 0);
    for (int i = 1; i <= N; i++) cin >> cost[i];

    vector<ull> reach(N + 2, 0);
    vector<int> nxt(N + 2, -1);

    deque<int> dq;

    for (int i = N; i >= 0; i--) {
        int j = i + 1;
        ull vj = cost[j] + reach[j];

        while (!dq.empty() && cost[dq.back()] + reach[dq.back()] > vj)
            dq.pop_back();
        dq.push_back(j);

        while (!dq.empty() && dq.front() > i + K)
            dq.pop_front();

        reach[i] = cost[dq.front()] + reach[dq.front()];
        nxt[i] = dq.front();
    }
}
```




```

cout << reach[0] << "\n";

vector<int> path;
int cur = 0;
while (cur != N + 1) {
    cur = nxt[cur];
    if (cur != N + 1) path.push_back(cur);
}

for (int i = 0; i < (int)path.size(); i++) {
    if (i) cout << " ";
    cout << path[i];
}
cout << "\n";

return 0;
}

```



Zadatak 5 - Penjanje

```
#include <vector>
#include <iostream>
#include <cmath>
#include <algorithm>

using namespace std;

vector<bool> visited(2000, false);

bool dfs(const vector<vector<int>> &adj,
        vector<int> &right_pair,
        vector<int> &left_pair,
        int u) {
    visited[u] = true;
    for (auto &v : adj[u]) {
        if (left_pair[v] == -1 ||
            (!visited[left_pair[v]] && dfs(adj, right_pair, left_pair, left_pair[v]))) {
            left_pair[v] = u;
            right_pair[u] = v;
            return true;
        }
    }
    return false;
}

int kuhn(int N, int M, const vector<vector<int>> &adj) {
    vector<int> right_pair(N, -1), left_pair(M, -1);
    for (int i = 0; i < N; ++i) {
        if (right_pair[i] != -1) continue; // Already paired.
        visited.assign(2000, false);
        dfs(adj, right_pair, left_pair, i);
    }
    int unmatched = 0;
    for (int i = 0; i < N; ++i) {
        if (right_pair[i] == -1) ++unmatched;
    }
    return unmatched;
}

/*
1. perform transitive closure using Floyd-Warshall
2. Construct bipartite graph with 2 copies of each vertex
```



```

3. Perform MCBM (Kuhn's algorithm) on the bipartite graph to get matching size M
4. If M = 0, answer is -1, else K = N - M + 1
*/

int main() {
    int N;
    cin >> N;
    vector<int> X(N), Y(N), S(N);
    for (int i = 0; i < N; ++i) {
        cin >> X[i] >> Y[i] >> S[i];
    }

    // Build adjacency matrix for direct reachability
    // adj[i][j] = true if you can move directly from hold i to hold j
    vector<vector<bool>> adj(N, vector<bool>(N, false));

    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            if (i == j) continue;

            // Can move from i to j if:
            // 1. Y[i] < Y[j] (must climb upwards)
            // 2. max(|X[i] - X[j]|, |Y[i] - Y[j]|) <= max(S[i], S[j])
            int dx = abs(X[i] - X[j]);
            int dy = abs(Y[i] - Y[j]);
            int max_dist = max(dx, dy);
            int max_friction = max(S[i], S[j]);

            if (Y[i] < Y[j] && max_dist <= max_friction) {
                adj[i][j] = true;
            }
        }
    }

    // Floyd-Warshall for transitive closure
    // After this, adj[i][j] = true if you can reach j from i (directly or indirectly)
    for (int k = 0; k < N; ++k) {
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                if (adj[i][k] && adj[k][j]) {
                    adj[i][j] = true;
                }
            }
        }
    }

    // Construct bipartite graph with 2 copies of each vertex

```



```

// Left side: vertices 0, 1, ..., N-1 (original vertices)
// Right side: vertices 0', 1', ..., (N-1)' (primed copies)
// Edge from i to j' if there's a path from i to j in the original graph
vector<vector<int>> bipartite_adj(N);

for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {
        if (i != j && adj[i][j]) {
            // Add edge from left vertex i to right vertex j
            bipartite_adj[i].push_back(j);
        }
    }
}

// Run MCBM (Kuhn's algorithm) on the bipartite graph
int unmatched = kuhn(N, N, bipartite_adj);
int matching_size = N - unmatched;

// Calculate and print answer
// If M = 0, answer is -1, else K = N - M + 1
int answer;
if (matching_size == 0) {
    answer = -1;
} else {
    answer = N - matching_size + 1;
}

cout << answer << endl;
return 0;
}

```

